SELECT · FROM · WHERE



Introduction to SQL

The Structured Query Language: core concepts, syntax, and worked examples for defining, querying, and managing relational databases.

DATABASE MANAGEMENT SYSTEMS · SQL FUNDAMENTALS

Mahe Zabin

WHAT WE'LL COVER

Agenda

01 Overview of SQL

02 SQL Data Definition

03 Basic Structure of SQL Queries

04 Additional Basic Operations

05 Set Operations

06 Null Values

07 Aggregate Functions

08 Nested Queries

What Is SQL?

- Structured Query Language — the standard language for creating, querying, and managing relational databases
- Originated as SEQUEL(**Structured English Query Language)**, developed by IBM in the 1970s for the research database project called System R.
- Standardized by ANSI in 1986 and ISO in 1987; the current standard is SQL:2016
- Combines four sub-languages in one unified language: DDL, DML, DCL, and TCL
- A declarative language — you describe WHAT result you want; the database engine decides HOW to compute it

THE FOUR SUB-LANGUAGES

DDL	Define & modify schema
DML	Query & modify data
DCL	Control access & permissions
TCL	Manage transactions

Why SQL Matters

- Non-procedural, set-at-a-time processing — a single statement operates on an entire set of rows at once
- Portable — the same core syntax works across MySQL, PostgreSQL, Oracle, SQL Server, and SQLite
- Complete — combines schema definition, querying, updates, security, and transaction control in one language
- Interfaces with host languages via embedded SQL, JDBC, ODBC, and dynamic SQL



SQL:2016

is the current ISO/IEC 9075 standard — and SQL remains the most widely used database language in production systems worldwide.

01

SECTION 1 OF 8

SQL Data Definition

Defining relations, data types, and integrity constraints with the Data Definition Language (DDL).



The Data Definition Language (DDL)

- Defines the schema for each relation (table): its name, attributes, and domains
- Specifies integrity constraints — primary key, foreign key, not null, check, unique
- Specifies indices maintained for each relation to speed up data access
- Specifies security and authorization information for each relation
- Specifies the physical storage structure of each relation on disk (implementation-specific)

CORE DDL COMMANDS

`CREATE`	Build a new database object
`ALTER`	Modify an existing object's structure
`DROP`	Permanently remove an object
`TRUNCATE`	Remove all rows, keep structure

Built-in Data Types in SQL

Data Type	Description	Example
char(n)	Fixed-length character string, padded with spaces	`char(10)`
varchar(n)	Variable-length character string, max length n	`varchar(50)`
int / integer	A whole number (machine-dependent size)	`int`
smallint	A small-range whole number	`smallint`
numeric(p,d)	Fixed-point number: p total digits, d after the decimal	`numeric(8,2)`
real, double precision	Floating-point and double-precision numbers	`real`
date	A calendar date (year, month, day)	`date '2026-07-28'`
time	A time of day (hour, minute, second)	`time '09:30:00'`
timestamp	A combined date and time value	`timestamp`

Defining a Relation — CREATE TABLE

- Every relation is created with a CREATE TABLE statement
- Each attribute is given a name and a domain (data type)
- Integrity constraints can be declared inline or listed at the end of the statement

GENERAL SYNTAX

```
CREATE TABLE r
  ( A1  D1,
    A2  D2,
    ...,
    An  Dn,
    <integrity-constraint 1>,
    ...,
    <integrity-constraint k> );
```

EXAMPLE

```
CREATE TABLE department (
  dept_name  varchar(20),
  building   varchar(15),
  budget     numeric(12,2),
  PRIMARY KEY (dept_name)
);
```

Integrity Constraints

Constraint	Meaning	Example
PRIMARY KEY	Uniquely identifies each tuple; implies NOT NULL + UNIQUE	`PRIMARY KEY (ID)`
FOREIGN KEY	Value must match a primary key value in a referenced table	`FOREIGN KEY (dept_name) REFERENCES department`
NOT NULL	The attribute may never take the null value	`name varchar(20) NOT NULL`
UNIQUE	No two tuples may agree on the listed attribute(s)	`UNIQUE (email)`
CHECK (P)	Every tuple must satisfy the predicate P	`CHECK (salary > 0)`

Modifying & Removing Relations

- ALTER TABLE adds, modifies, or drops a column on an existing relation
- DROP TABLE permanently removes a relation's structure and all of its data
- DELETE FROM removes only the rows — the table structure remains intact

SYNTAX

```
ALTER TABLE r ADD A D;
-- add new attribute A of domain D

ALTER TABLE r DROP COLUMN A;
-- remove attribute A

DROP TABLE r;
-- delete the relation entirely
```

EXAMPLE

```
ALTER TABLE instructor
  ADD phone varchar(15);

ALTER TABLE instructor
  DROP COLUMN phone;

DROP TABLE temp_grades;
```

02

SECTION 2 OF 8

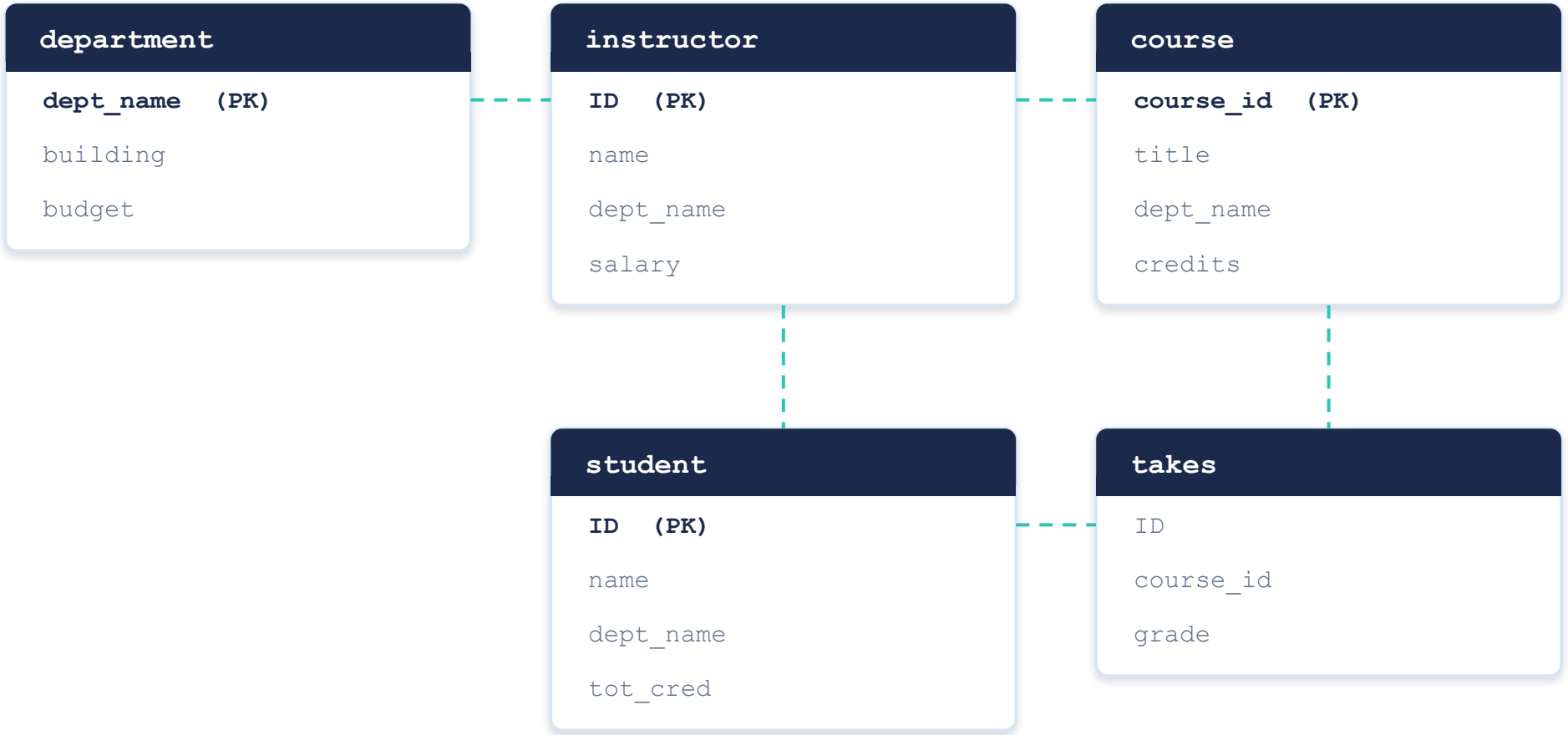
Basic Structure of SQL Queries

The SELECT–FROM–WHERE building block that underlies almost every SQL query.



The Example Database Schema

This university schema is used for every example throughout the rest of this deck.



SELECT – FROM – WHERE

The fundamental form of an SQL query has three clauses: SELECT, FROM, and WHERE

GENERAL FORM

```
SELECT  A1, A2, ..., An
FROM    r1, r2, ..., rm
WHERE   P;

-- equivalent to relational algebra:
-- Π A1,...,An ( σ P ( r1 × r2 × ... × rm ) )
```

The FROM Clause & Combining Relations

- The FROM clause specifies **which table(s) SQL should retrieve data from**.
- FROM lists the relations used by a query: If **more than one table** is listed, SQL first creates a **Cartesian Product (Cross Join)** before applying any conditions
- Every tuple of the first relation is paired with every tuple of the second
- WHERE is used to restrict this product to matching tuples, forming an implicit join
- An attribute name common to two relations must be qualified as relation.attribute

EXAMPLES

```
-- Cartesian product of instructor and teaches
SELECT * FROM instructor, teaches;

-- join: instructors with the course IDs they teach
SELECT name, course_id
FROM   instructor, teaches
WHERE  instructor.ID = teaches.ID;
```

The SELECT Clause

- The SELECT clause specifies **which column(s) should be displayed** in the query result.It is used to retrieve one, multiple, or all columns from a table.
- SQL names are case-insensitive (this deck uses lowercase for keywords and identifiers)

- Query results may contain duplicate rows by default — use DISTINCT to eliminate them
- ALL is the default (retain duplicates); it is rarely written explicitly
- SELECT can include arithmetic expressions over constants and attributes
- The asterisk * denotes "all attributes of the FROM relations"

EXAMPLES

```
SELECT name FROM instructor;

SELECT DISTINCT dept_name FROM instructor;

SELECT ID, name, salary/12 AS monthly_pay
FROM   instructor;
```

The WHERE Clause

- Corresponds to the selection predicate of relational algebra which means it is used to **select rows that satisfy a condition**
- Comparison operators: `=` `<>` `<` `>` `<=` `>=`
- Logical connectives AND, OR, and NOT combine multiple conditions
- Comparisons may include arithmetic expressions, not just plain attributes

EXAMPLE

```
-- instructors in Comp. Sci. earning above 70000
SELECT name
FROM   instructor
WHERE  dept_name = 'Comp. Sci.' AND salary > 70000;
```

SELECT – FROM – WHERE

```
SELECT Name, CGPA
FROM Student
WHERE Department = 'AI';
```

StudentID	Name	Department	CGPA
101	Alice	CSE	3.90
102	John	AI	3.60
103	Emma	CSE	3.80
104	Sophia	AI	3.95

How SQL Works

1. FROM → Choose the table to search.

FROM Student

2. WHERE → Filter the rows that satisfy the condition.

WHERE Department = 'AI'

SELECT – FROM – WHERE

Remaining rows:

Name	Department	CGPA
John	AI	3.60
Sophia	AI	3.95

3.SELECT → Display only the required columns.

SELECT Name, CGPA

Final Output

Name	CGPA
John	3.60
Sophia	3.95

Easy to Remember:
FROM = Where to search → **WHERE** = Which rows → **SELECT** = What to display

03

SECTION 3 OF 8

Additional Basic Operations

Renaming, pattern matching, ordering, and range/membership tests that refine a query.



The Rename Operation — AS

- AS renames a relation and/or an attribute in the query result
- Useful for readability, self-joins (comparing a relation to itself by joining a table with itself), and naming computed columns
- old-name AS new-name works on both attributes and relations; AS is optional in most systems

EXAMPLES

```
-- rename an output column
SELECT ID, name AS instructor_name FROM instructor;

-- self-join: instructors paid more than any
-- Biology instructor (tuple variables T and S)
SELECT DISTINCT T.name
FROM   instructor AS T, instructor AS S
WHERE  T.salary > S.salary AND S.dept_name = 'Biology';
```

Renaming an Output Column (AS): The AS keyword changes the column name only in the query result.It does not change the original table.

Renaming a Table (AS) – Table Alias: Here, SQL is comparing one instructor with another instructor.Since both instructors are stored in the same table, SQL creates two temporary names (aliases) for the table.

The Rename Operation — AS

Understanding `WHERE T.salary > S.salary` (Self-Join)

Example: Instructor Table

Name	Department	Salary
Alice	Biology	70000
John	Computer Science	90000
Emma	Biology	65000
David	Physics	75000

SQL Query

```
SELECT DISTINCT T.name
FROM instructor AS T, instructor AS S
WHERE T.salary > S.salary
AND S.dept_name = 'Biology';
```

The Rename Operation — AS

How It Works

Step 1: SQL creates **two temporary copies** of the same table

T (Any Instructor)	S (Biology Instructor)
John (90000)	Alice (70000)
David (75000)	Emma (65000)

Step 2: Compare the salaries.

John's salary (90000) > Alice's salary (70000)
David's salary (75000) > Emma's salary (65000)

Since the condition is **TRUE**, SQL displays **T.name**

Final Output

Name
John
David
Alice

T and **S** are **two temporary names (aliases) for the same table**. SQL compares the salary of one instructor (**T**) with another instructor (**S**) from the same table.

String Operations — LIKE

- String literals are written in single quotes, e.g. 'Comp. Sci.' It is a text value enclosed in **single quotes (' ')**
- LIKE matches a string against a pattern using two special characters: % matches any substring (zero or more characters); _ matches exactly one character
- ESCAPE lets you search for a literal % or _ character
- NOT LIKE excludes matching rows

EXAMPLES

```
-- names beginning with 'Ada'
SELECT name FROM instructor WHERE name LIKE 'Ada%';

-- names containing the substring 'dar'
SELECT name FROM instructor WHERE name LIKE '%dar%';
```

String Operations — LIKE

1. String Literals

Example:

```
SELECT *
FROM department
WHERE dept_name = 'Comp. Sci.';
```

✔ `'Comp. Sci.'` is a string literal

2. LIKE Operator

Wildcard	Meaning	Example
%	Matches **zero or more characters**	`'A%'` → Alice, Adam, Alex
_	Matches **exactly one character**	`'A_'` → Al, An (not Alex)

Example:

```
SELECT *
FROM instructor
WHERE name LIKE 'A%';
```

Result: All instructors whose names start with **A**

3. ESCAPE

Example:

```
SELECT *
FROM product
WHERE product_name LIKE '%\%%' ESCAPE '\';
```

✔ Finds product names containing the **%** symbol, such as **"50% Discount"**

4. NOT LIKE

Example:

```
SELECT *
FROM instructor
WHERE name NOT LIKE 'A%';
```

Result: All instructors **whose names do not start with 'A'**

Ordering, BETWEEN & IN

- ORDER BY sorts results by one or more attributes — ascending (ASC, default) or descending (DESC)
- BETWEEN x AND y is shorthand for >= x AND <= y
- IN (v1, v2, ...) tests membership in an explicit list of values; NOT IN excludes them

EXAMPLES

```
-- order instructors by salary, highest first
SELECT name, salary FROM instructor ORDER BY salary DESC;

-- salary within a range
SELECT name FROM instructor
WHERE  salary BETWEEN 60000 AND 90000;

-- department is one of a list
SELECT name FROM instructor
WHERE  dept_name IN ('Physics', 'Music');
```

Ordering, BETWEEN & IN

1. ORDER BY

`ORDER BY` is used to **sort the query result** based on one or more columns.

- **ASC** (Ascending) → Smallest to largest, A to Z *(default)*
- **DESC** (Descending) → Largest to smallest, Z to A

Example

```
SELECT name, salary
FROM instructor
ORDER BY salary DESC;
```

Result: Instructors are displayed from the **highest salary to the lowest salary**

2. BETWEEN

`BETWEEN x AND y` selects values **within a specified range (inclusive)**.

It is equivalent to:

salary >= x AND salary <= y

Example

```
SELECT name, salary

FROM instructor
WHERE salary BETWEEN 60000 AND 80000;
```

Result: Displays instructors whose salary is **between $60,000 and $80,000**, including both **$60,000** and **$80,000**

Ordering, BETWEEN & IN

3. `IN`

Example

```
SELECT name
FROM instructor
WHERE dept_name IN ('Biology', 'Physics');
```

Result: Displays instructors from the **Biology** or **Physics** departments

`NOT IN`

`NOT IN` returns rows whose values **are not in the specified list**.

Example

```
SELECT name
FROM instructor
WHERE dept_name NOT IN ('Biology', 'Physics');
```

Result: Displays instructors from **all other departments** except Biology and Physics.

04

SECTION 4 OF 8

Set Operations

Combining the results of two queries using UNION, INTERSECT, and EXCEPT.



Set Operations — UNION

- SQL set operations combine the results of two SELECT queries, treating each as a set (or multiset) of rows
- Union-compatibility rule: both queries must return the same number of attributes , with compatible domains means Both queries must return **the same number of columns. Example:**
  SELECT Name, Salary
  FROM Instructor

  UNION

  SELECT Name, Salary
  FROM Employee;

  Both queries return **2 columns**, so the query is valid

```
-- courses taught in Fall 2017, Spring 2018, or both
(SELECT course_id FROM section
 WHERE semester = 'Fall' AND year = 2017)
UNION
(SELECT course_id FROM section
 WHERE semester = 'Spring' AND year = 2018);
```

Result of the first query: Returns the **course IDs** of all courses offered in **Fall 2017**.

Result of the second query: Returns the **course IDs** of all courses offered in **Spring 2018**. UNION then combines both results and **removes duplicate course IDs**.

Set Operations — UNION

UNION returns rows appearing in either result and automatically eliminates duplicates; A row appears only once, even if it exists in both queries and UNION ALL retains duplicates — faster, since it skips duplicate elimination

Example:

Instructor Table

Name
Alice
John

Student Table

Name
John
Emma

SQL Query

```
SELECT Name
FROM Instructor

UNION

SELECT Name
FROM Student;
```

Step 1: Combine Both Results

Alice
John
John
Emma

Notice that **John appears twice**

Step 2: Remove Duplicates

`UNION` automatically removes duplicate rows.

Final Result

Name
Alice
John
Emma

INTERSECT & EXCEPT

- INTERSECT returns rows appearing in BOTH query results (duplicates removed)
- EXCEPT — MINUS in Oracle — returns rows in the first result that do NOT appear in the second
- Both follow the same union-compatibility rule as UNION

INTERSECT — TAUGHT IN BOTH TERMS

```
(SELECT course_id FROM section
 WHERE semester='Fall' AND year=2017)
INTERSECT
(SELECT course_id FROM section
 WHERE semester='Spring' AND year=2018);
```

EXCEPT — FALL ONLY, NOT SPRING

```
(SELECT course_id FROM section
 WHERE semester='Fall' AND year=2017)
EXCEPT
(SELECT course_id FROM section
 WHERE semester='Spring' AND year=2018);
```

INTERSECT & EXCEPT

Fall 2017 Courses

Course_ID
CS101
CS102
CS103

Spring 2018 Courses

Course_ID
CS102
CS103
CS104

INTERSECT Result:

Course_ID
CS102
CS103

Except Result:

Course_ID
CS101

✔ `CS101` was offered in **Fall 2017 only**, so it is returned as `EXCEPT` returns rows that appear in the **first query** but **not in the second query**.

05

SECTION 5 OF 8

Null Values

Representing unknown or missing data, and the three-valued logic it introduces.



Understanding NULL

- NULL represents a value that is unknown or does not exist — it is NOT the same as zero or an empty string
- Any arithmetic expression involving NULL evaluates to NULL, e.g. 5 + NULL = NULL
- Any comparison with NULL (=, <>, <, >, ...) evaluates to unknown — never true or false
- Because of this, regular comparison operators cannot be used to test for NULL

EXAMPLE

```
SELECT salary + NULL FROM instructor;  -- always NULL

SELECT * FROM instructor WHERE salary = NULL;
-- returns NOTHING — this is a common mistake!
```

Testing for NULL & Three-Valued Logic

- Use IS NULL / IS NOT NULL to correctly test for null values
- **IS NULL is** used to find rows where a column has **no value**.
- **IS NOT NULL is** used to find rows where a column **contains a value**.
- SQL uses three-valued logic: true, false, and unknown
- A WHERE clause keeps only rows where the predicate evaluates to true — unknown rows are excluded

EXAMPLES

```
SELECT name FROM instructor
WHERE  salary IS NULL;

SELECT name FROM instructor
WHERE  dept_name IS NOT NULL;
```

Result: Displays instructors whose **salary is missing (NULL) and** Displays dept_name that **is available.**

AND / OR TRUTH TABLE

P	Q	P AND Q	P OR Q
true	unknown	unknown	true
false	unknown	false	unknown
unknown	unknown	unknown	unknown

Example:

Suppose `salary = NULL`

SQL Query:
salary > 50000

Result:

UNKNOWN

because SQL doesn't know the salary.

Testing for NULL & Three-Valued Logic

Imagine

Is Alice taller than 170 cm?

Case 1

Alice's height is **180 cm**. Answer: **TRUE**

Case 2

Alice's height is **160 cm**. Answer: **FALSE**

Case 3

Alice's height is **unknown** because nobody measured it. Can you answer: Is Alice taller than 170 cm? You **can't say Yes**. You **can't say No**.The only honest answer is **I don't know.**

In SQL,"I don't know" is called: **UNKNOWN**

Testing for NULL & Three-Valued Logic

Suppose we have this table.

Name	Salary
Alice	70000
John	NULL
Emma	50000

Now SQL checks

salary > 60000

Alice Salary =70000

Check: 70000 > 60000

Result: TRUE

Emma Salary=50000

Check: 50000 > 60000

Result: FALSE

John Salary=NULL

Check: NULL > 60000

Can SQL answer?No. Because SQL doesn't know John's salary.

Maybe 70000 Then the answer would be TRUE OR Maybe 30000 Then the answer would be FALSE.

Since SQL doesn't know,the answer is**UNKNOWN**

06

SECTION 6 OF 8

Aggregate Functions

Summarizing data with COUNT, SUM, AVG, MIN, MAX, GROUP BY, and HAVING.



Aggregate Functions

- Aggregate functions take a collection of values as input and return a single summary value
- COUNT(*) counts rows; COUNT(A) counts non-null values of attribute A
- SUM(A) totals a numeric column; AVG(A) computes its average
- MIN(A) / MAX(A) return the smallest / largest value
- DISTINCT inside an aggregate considers only unique values, e.g. COUNT(DISTINCT dept_name)

EXAMPLES

```
SELECT AVG(salary) FROM instructor
WHERE  dept_name = 'Comp. Sci.';

SELECT COUNT(DISTINCT dept_name) FROM instructor;
SELECT MAX(salary), MIN(salary) FROM instructor;
```

Answer: Returns the **average salary of all instructors in the Computer Science department.**

Answer: Returns the **total number of different (unique) departments** in the `instructor` table.`DISTINCT` ensures that each department is counted **only once**.

Answer: Returns the **highest salary** and the **lowest salary** among all instructors.

The GROUP BY Clause

- GROUP BY partitions the tuples of a relation into groups that share the same value(s) on given attribute(s)
- Aggregate functions are then applied independently to each group
- Every attribute in the SELECT list must either appear in GROUP BY or be inside an aggregate function

EXAMPLE

```
-- average salary in each department
SELECT dept_name, AVG(salary) AS avg_salary
FROM   instructor
GROUP BY dept_name;
```

Answer: Returns the **average salary for each department** and displays the result with the column name **`avg_salary for example: Biology 7500, Computer science 95000`**.

The HAVING Clause

- **HAVING filters groups after GROUP BY, while WHERE filters individual rows before GROUP BY.**
- **HAVING can use aggregate functions such as AVG(), COUNT(), SUM(), MAX(), and MIN().**
- Evaluation order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

EXAMPLE

```
-- departments with average salary above 42000
SELECT dept_name, AVG(salary) AS avg_salary
FROM   instructor
GROUP BY dept_name
HAVING AVG(salary) > 42000;
```

Answer: Returns **only the departments whose average salary is greater than 42,000**, along with their average salary (`avg_salary`).

07

SECTION 7 OF 8

Nested Queries

Subqueries in WHERE, SELECT, and FROM — for set membership, comparison, and existence tests.



Nested Queries & Set Membership

- A subquery is a SELECT–FROM–WHERE expression nested inside another query — usually in WHERE, FROM, or SELECT
- IN / NOT IN test membership in the set of values returned by a subquery
- Subqueries express complex conditions that would be awkward to write as a single flat query

EXAMPLES

```
-- course_ids taught in both Fall 2017 and Spring 2018
SELECT DISTINCT course_id FROM section
WHERE  semester='Fall' AND year=2017 AND course_id IN
  (SELECT course_id FROM section
   WHERE semester='Spring' AND year=2018);
```

The inner query (subquery) finds all course IDs offered in Spring 2018.
The outer query checks the Fall 2017 courses.
`IN` keeps only the course IDs that also appear in the Spring 2018 list.

Set Comparison & EXISTS

- > SOME / < SOME / = SOME (or ANY) is true if the comparison holds for at least one subquery value
- > ALL / < ALL is true if the comparison holds for every subquery value
- EXISTS tests whether a subquery's result is non-empty; NOT EXISTS tests for emptiness
- EXISTS subqueries are often correlated — they reference a column from the outer query

ALL — HIGHEST-PAID OUTSIDE BIOLOGY

```
SELECT name FROM instructor
WHERE  salary > ALL
  (SELECT salary FROM instructor
   WHERE dept_name='Biology');
```

Retrieves the names of instructors from the `instructor` table. Retrieves the salaries of all instructors in the Biology department. Returns the names of instructors who earn more than every instructor in the Biology department

Subqueries in the FROM Clause

- A subquery expression can be used anywhere a relation could appear — including in FROM
- The subquery's result is treated as a temporary, derived relation and must be given an alias
- This is useful for computing an intermediate aggregate result and then querying it further

EXAMPLE

```
-- departments whose average salary exceeds 42000
SELECT dept_name, avg_salary
FROM  (SELECT dept_name, AVG(salary) AS avg_salary
       FROM   instructor
       GROUP BY dept_name) AS dept_avg
WHERE avg_salary > 42000;
```

Inner Query (Subquery): Calculates the average salary for each department and names the average column `avg_salary`.
Outer Query: Retrieves the department name and its average salary from the subquery result.
WHERE Clause: Keeps only the departments whose average salary is greater than 42,000.
Finally returns the names of departments with an average salary greater than 42,000, along with their average salary.

WRAP-UP

Key Takeaways

1. SQL combines DDL, DML, DCL & TCL into one declarative language
2. CREATE TABLE defines schema, data types & integrity constraints
3. SELECT – FROM – WHERE is the core query building block
4. AS, LIKE, ORDER BY, BETWEEN & IN refine query results
5. UNION, INTERSECT & EXCEPT combine query results as sets
6. NULL needs IS NULL / IS NOT NULL and three-valued logic
7. COUNT, SUM, AVG, MIN, MAX with GROUP BY / HAVING summarize data
8. Nested queries (IN, ANY/ALL, EXISTS, FROM-subqueries) express complex conditions